

um-game-playing-strength-of-mcts-variants

10th place solution

Rank	10
Team	DO THE HARLEM SHAKE
Author	Taha_Alshatiri
Topic ID	549605
Version	Original

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549605>

Retrieved with Kaggle CLI on 2026-07-03.

Hello everyone, am very excited to share the solution that got us into the 10th position, before I start I want to thank my teammates [@jawadkc66](#) [@mohammedkhurd](#) [@yuanzhezhou](#)

And we want to thank [@yunsuxiaozi](#) for his public notebooks [Mcts|starter](#) it helped a lot and was the notebook that we build our baseline upon

What worked:

1)Data generation(augmentation):

we saw since the beginning that extra data would most likely bring a better result, but I didn't have the computation to do it and pseudo labelling didn't work, probably because the error was very high(>0.4) so predicting fake data with this error rate and treating it as ground truth won't be a good idea, so the idea I got is augmenting the dataset and doubling it by flipping the agent order, taking the compliment of the advantagep1 and taking the negation of utility_agent1, as shown here

```
augmented_df = train.copy()
augmented_df = augmented_df.rename(columns={
    : ,
    :,
})

augmented_df[] = - augmented_df[]
augmented_df[] = -augmented_df[]
train = pd.concat([train, augmented_df], ignore_index=)
```

this simple trick just pushed the public lb of MCTS|starter notebook from 0.427 to 0.422 and it placed us in the top 20 at that time when I was going as a dou with [@mohammedkhurd](#) I think that this our best idea

2)advp1 binning

I noticed also that advp1 was the most important feature but the way it is collected makes it a bit noisy so I got the idea of binning it to 10 numbers and it showed a good improvement, but I didn't remove the original feature having them both seem to give the best performance, my friends [@jawadkc66](#) [@mohammedkhurd](#) tried other numbers for binning but 10 seems to give the best performance. This increased the cv and increased lb from 0.422 to 0.420

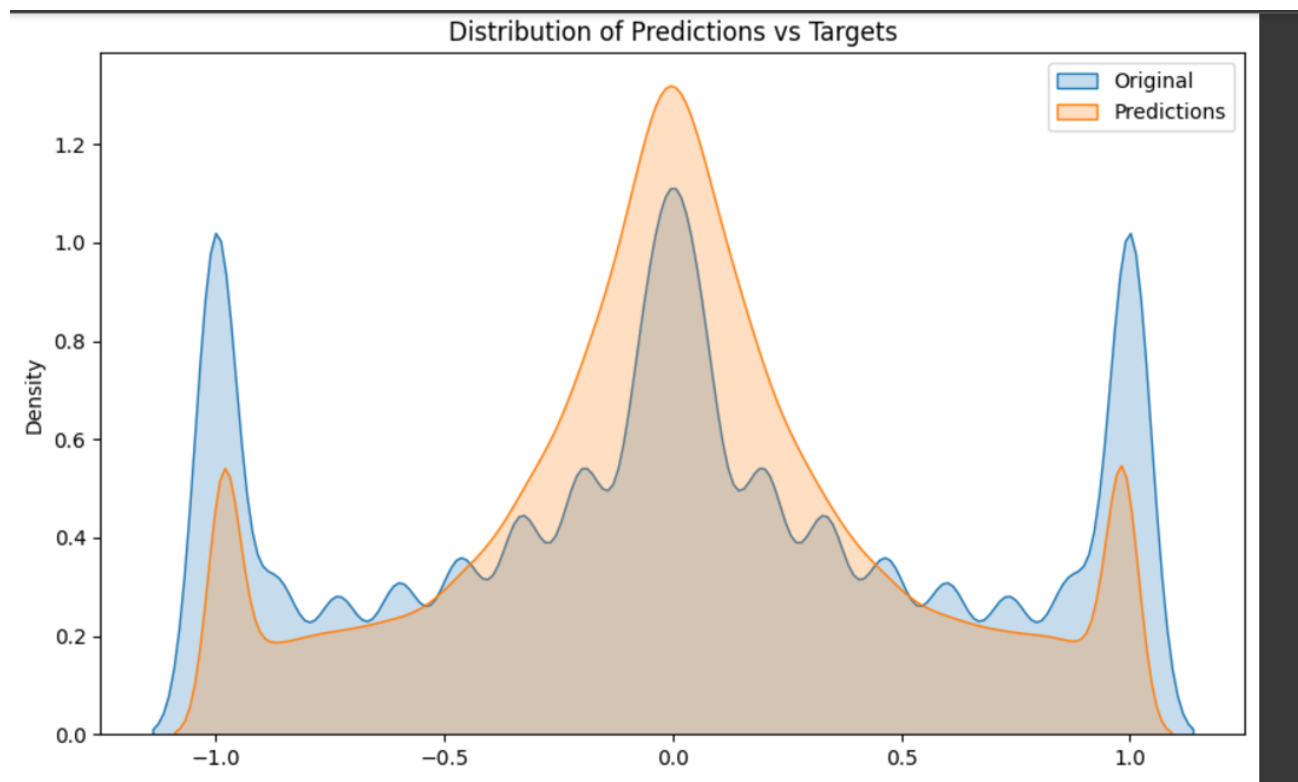
3)increasing folds

Our teammate [@yuan](#) suggested introducing more folds for more diversity ensembling so we increased it from 5 to 10, this pushed the lb score further to 0.419 which actually mattered

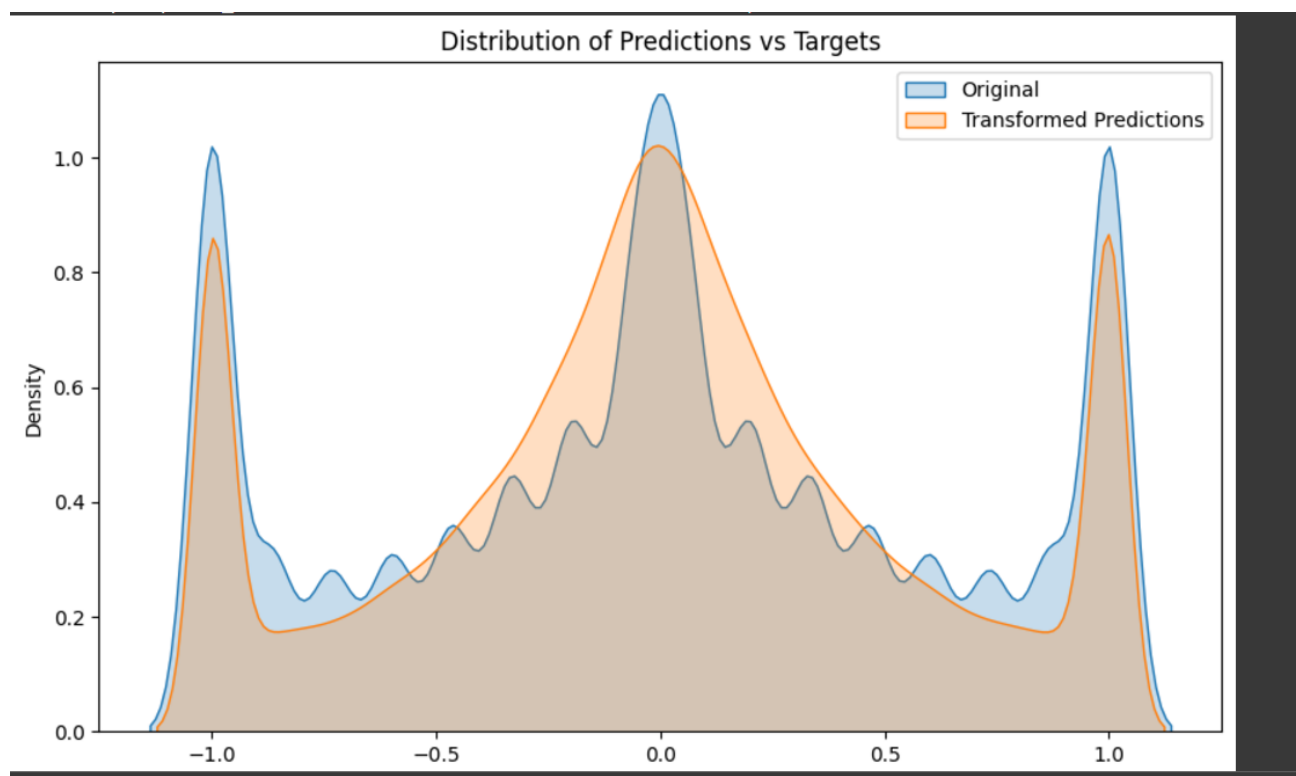
4)multiplying predictions by a constant(overfitting)

In the public notebook we used the predictions was getting multiplied by 1.1, which is risky and overfitting, but when I visualize the model predictions on the train I noticed our catboost seems to always squeeze the predictions close to zero, so multiplying the predictions by a number seems to decrease the kl divergence and make the prediction-target distributions more closer here are some visualization to make it more clear

predictions-target



predictions*1.3-target



so basically multiplying the predictions with a number seems to make the score better but never tried it even visualization shows that it worked good, but our friend [@yuanzhezhou](#) did it and it improved the lb from 0.419 to 0.417, he did twice, once with 1.25 and once with 1.3 we choose the 1.25 part but both gave the exact same lb and private and public results , this is what made us jump from silver to gold

these where our main ideas nothing else to mention except some parameter hyperparameter tuning maybe

What didn't work?

if I want to list all the tricks that I tried and didn't work it would've took me a whole book to note it, same for my teammates but here are some of what I tried and didn't work

- 1)pseudo labelling(its clear why)
- 2) creating advp2 and making some interactions such as advp1/advp2 and else, could be that the gdbt already handle these interactions maybe higher interactions and more feature engineering could've made us get an advantage from it
- 3)nn models(me, [@yuanzhezhou](#) and [@jawadkc66](#) tried it, it just seem to never work for us even when they give good cv)

what we could've done

- 1)deleting some gamerulesets: there could be some very noisy gamerulesets that doesn't even exist in test so trying to delete it could've improved the lb and am pretty sure there are some mistakes in the data for some gamerulesets and i might talk about it in another discussion but i couldn't benefit from this point

2)better modelling: I think that we just didn't spend enough time on modelling we tried our tricks on public notebook but we didn't make a solid baseline, we tried a bit but it seems that we couldn't outperform the public notebook we used

3)more feature engineering: Simple as it is

thank you everyone, hope that you meet me again in another competition as a competition expert :)